

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	G.Navya Sri	Reg. No.:	192524170
Section / Batch:	B.Tech AI&DS	Date:	20-07-2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:			100 Marks

SECTION A – APPLICATION BASED QUESTIONS

Q1. Hospital Patient Record System

A hospital system compares every patient record with every other record to detect duplicate entries using nested loops. Recall what nested loops imply in complexity analysis, explain why this approach increases execution time, and determine both the time complexity and space complexity using asymptotic notation.

Q2. Digital File Search System

A file search tool repeatedly reduces the search space by half while locating a file in a sorted directory. Define logarithmic growth, explain how halving affects the number of steps, and determine both the time complexity and space complexity of this algorithm.

Q1. Hospital Patient Record System

1. Understanding of Problem Context

The hospital system compares every patient record with every other patient record to identify duplicate entries. This requires checking each record against all remaining records using **nested loops**. As the number of patient records increases, the number of comparisons grows rapidly, making the program slower.

2. Application of Concepts

Nested loops mean that one loop runs completely for every iteration of another loop. If there are n patient records:

- The outer loop executes n times.
- The inner loop also executes approximately n times for each outer iteration.
- Therefore, the total number of comparisons is approximately $n \times n = n^2$.

Since no significant extra data structure is used (only a few variables for looping and comparison), the additional memory required remains constant.

3. Problem-Solving Approach

1. Start with the first patient record.
2. Compare it with every other patient record.
3. Move to the next patient record.
4. Repeat until every possible pair has been checked.
5. If matching details are found, identify them as duplicate records.

4. Accuracy of Solution

- **Time Complexity:** $O(n^2)$ because every record is compared with every other record.
- **Space Complexity:** $O(1)$ because only a constant amount of extra memory is required.

5. Clarity

Using nested loops significantly increases execution time because the number of comparisons grows quadratically as the number of records increases. While this method correctly detects duplicates, it becomes inefficient for large datasets due to its $O(n^2)$ time complexity.

Q2. Digital File Search System

1. Understanding of Problem Context

The file search system searches for a file in a **sorted directory** by repeatedly reducing the search space by half. This is the principle of **binary search**, which is much faster than checking every file one by one.

2. Application of Concepts

Logarithmic growth means that the number of operations increases very slowly as the input size increases. In binary search:

- Compare the target file with the middle element.
- If the target is smaller, search the left half.
- If the target is larger, search the right half.
- Repeat until the file is found or the search space becomes empty.

Since the search space is halved after each comparison, the number of steps is proportional to $\log_2(n)$.

3. Problem-Solving Approach

1. Find the middle file in the sorted directory.
2. Compare it with the target file.
3. If it matches, stop.
4. Otherwise, eliminate half of the remaining files.
5. Repeat the process on the remaining half until the file is found or no files remain.

4. Accuracy of Solution

- **Time Complexity:** $O(\log n)$ because the search space is reduced by half in every step.
- **Space Complexity:** $O(1)$ for the iterative version, as only a few variables are used.

5. Clarity

Halving the search space greatly reduces the number of comparisons needed, making binary search highly efficient for large sorted datasets. Therefore, the algorithm has $O(\log n)$ time complexity and $O(1)$ space complexity (iterative implementation).

Code of Conduct

I G Navya Sri (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G Navya Sri

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

21/7/26

CO₂ - AT₁
Analytical Questions
Solving!

Name : Navya Bri. G
Reg no: 192524170
Subject : Design Analysis
of Algorithms
Subject code : CNA0614

1. Binary Search - Target Near upper middle region.

Given the sorted dataset [2, 6, 10, 14, 18, 22, 26, 30, 34], The task is to find the element 22 using binary search algorithm. Since the data is arranged in ascending order. Binary search is the most efficient searching technique.

Dataset : [2, 6, 10, 14, 18, 22, 26, 30, 34]

target element = 22

⇒ low = 0

⇒ high = 8

⇒ mid = $(0+8)/2 = 4$

⇒ Middle Element = 18.

Comparison

⇒ $22 > 18$

⇒ Search the right half

Iteration 2

$$\Rightarrow \text{low} = 5$$

$$\Rightarrow \text{high} = 8$$

$$\Rightarrow \text{mid} = (5+8)/2 \\ = 6$$

$$\Rightarrow \text{middle element} = 26$$

comparision

$\Rightarrow$ search the left half

$\Rightarrow$ New Search Interval: $[22]$

Iteration 3

$$\Rightarrow \text{low} = 5$$

$$\Rightarrow \text{high} = 5$$

$$\Rightarrow \text{mid} = 5$$

$$\Rightarrow \text{middle element} = 22$$

comparision

$$\Rightarrow 22 = 22$$

$\Rightarrow$ element found.

Binary search table

Iteration	low	high	mid	middle Element	comparision	Result
1	0	8	4	18	$22 > 18$	search right half
2	5	8	6	26	$22 < 26$	search left half
3	5	5	5	22	$22 = 22$	element found